

Contents

1	DRDG3D mixed-flux exploration — what does the reference do that SEAS-MFEM may be missing at $p \geq 2$?	1
1.1	Overview	2
2	OUTCOME & ROOT CAUSE (revised 2026-05-28)	2
2.1	TL;DR of the comparison	2
2.2	1. The controlled isolation — what we now KNOW (vs the 2026-04-28 hypotheses)	2
2.3	2. Code-verified kernel agreement (the flux is NOT the discrepancy)	3
2.4	3. ROOT CAUSE — imaginary-axis stability of the time integrator $\times$ zero dissipation	3
2.5	4. This corrects Candidate 1’s framing	4
2.6	5. Re-adjudication of the five 2026-04-28 candidates	4
2.7	6. Remedies, re-ranked by the root cause	5
2.8	7. Bottom line of the comparison	6
2.9	Architecture	6
2.10	Detailed walkthrough — DRDG3D’s central-flux dispatch	7
2.11	Call graph (DRDG3D mixed-flux path, ADER-equivalent)	9
2.12	Conventions	10
2.13	Gotchas	10
2.14	Differential analysis — what is SEAS-MFEM doing differently?	10
2.15	Open questions	12
2.16	Recommended next steps for the SEAS-MFEM TPV104 instability	12

1 DRDG3D mixed-flux exploration – what does the reference do that SEAS-MFEM may be missing at $p \geq 2$?

Date: 2026-04-28 **Source:** /Users/chunhuizhao/projects/drdg3d (Wenqiang Zhang, MIT-licenced reference for the Zhang et al. 2023 mixed-flux DG dynamic-rupture paper) **Motivation:** Our `--mixed-flux adjacent --order 2 --ader-order 3` TPV104 production run (job 7680797) develops point-source spurious slip during rupture propagation; the same configuration with `--mixed-flux none` (job 7680798) is clean. We need to know what DRDG3D — the published reference for the Zhang 2023 dispatch — does that we don’t.

► **UPDATE 2026-05-28 — comparison outcome added (this doc was moved to document/mixed_flux_dev/).** Since this 2026-04-28 audit, a *controlled single-variable isolation* (SAFS LSW: job 7751974 `--mixed-flux none` vs job 7751975 `--mixed-flux adjacent`, identical mesh/config/nucleation) plus a direct re-reading of both codes have **resolved the comparison**. Bottom line: the central-flux *kernel* is algebraically identical to DRDG3D’s and is **not** the bug; the blow-up is the interaction of the central flux’s zero dissipation with the **time integrator**, and the decisive quantity is the integrator’s stability on the **imaginary axis**. The hypothesis-era candidate list below (§Differential analysis) is kept as the supporting trail, with each verdict updated. See “**OUTCOME & ROOT CAUSE (revised 2026-05-28)**” immediately below. Companion evidence: the 2026-

1.1 Overview

DRDG3D solves 3D linear elastodynamics + dynamic rupture with a nodal discontinuous-Galerkin method on tetrahedral meshes. The same mixed- flux dispatch concept (central flux on faces near the fault, upwind elsewhere) lives in `src/mod_wave.F90`. Reading the source against SEAS-MFEM's `MixedFluxMode::Adjacent` path turns up four substantive differences, only one of which is a likely culprit for our blow-up.

2 OUTCOME & ROOT CAUSE (revised 2026-05-28)

2.1 TL;DR of the comparison

The central-flux **kernel is not the bug**. SEAS-MFEM's `GodunovFlux::Central` ($F^* = \frac{1}{2} \cdot A_n \cdot (Q^- + Q^+)$) is algebraically identical to DRDG3D's `fstar =  $\frac{1}{2} \cdot (F_R - F_L)$` central lift (`mod_wave.F90:587-602`), verified term-by-term, and **both codes keep the fault face itself on the upwind/friction-Riemann path** (never central). The blow-up is the interaction of the central flux's *zero dissipation* with the **time integrator**. The single decisive discrepancy:

SEAS-MFEM advances the central-flux faces with an ADER / Cauchy–Kovalevskaya (Taylor-in-time) one-step map of order `ader_order` (= 2 for every SAFS run, `spatial_friction.hpp:94`); DRDG3D advances with a 4th-order, 5-stage low-storage Runge–Kutta (RK54). Central flux makes the affected modes purely *imaginary* (energy-neutral operator). The ADER-O2 stability function amplifies *every* imaginary mode at *any* Δt ; RK54's stability region *contains* a segment of the imaginary axis. Same flux kernel, opposite fate.

2.2 1. The controlled isolation – what we now KNOW (vs the 2026-04-28 hypotheses)

A single-variable experiment (full data: the 2026-05-26 companion doc) settles it. Same triq mesh, same LSW Dc2 no-cap config, same `gradual overstress` nucleation; **only `--mixed-flux` changed**:

job	<code>--mixed-flux</code>	central faces	<code>dt_cfl</code> (s)	result
7751974	none (upwind)	0	4.18×10^{-4}	STABLE — $V_{\max}$ 10.5 m/s at $t=15.2$ s, $\max_{\text{slip}}$ 27.5 m
7751975	adjacent	241 089	3.76×10^{-4} (smaller)	RUNAWAY — $V_{\max}$ 8.9×10^6 m/s, $\max_{\text{slip}}$ 453 km by $t=13.2$ s

Two facts from this isolation are the fingerprints of the root cause: 1. **The flux is the only variable**, and it is the difference between a clean rupture and a 453 km-slip catastrophe. 2. **The adjacent run used a smaller Δt ($0.9 \times \text{CFL}$) and still blew up $\Rightarrow$ “not a CFL trip.”** This is the diagnostic signature explained rigorously in §3.

2.3 2. Code-verified kernel agreement (the flux is NOT the discrepancy)

Read directly in both sources (2026-05-28):

- **SEAS** (`godunov_flux.hpp:64–68`, pinned by `test_godunov_central_flux`): Interior — Central = $+\frac{1}{2} \cdot |A_n| \cdot (Q^- - Q^+)$, with $|A_n| = T \cdot (A^+ - A^-) \cdot T^{-1} \succeq 0$ (eigenvalues $|\pm c_p|, |\pm c_s|$). So **Central is the upwind flux with the *entire* dissipation term deleted.**
- **DRDG3D** (`mod_wave.F90:591–601` then `goto 100`): the central block sets $fstar = \frac{1}{2} \cdot (F_R - F_L)$ and **skips** the impedance-weighted upwind/a block (`:675–688`, $\alpha_1 = (\Delta T_n + Z_p^{\text{out}} \cdot \Delta v_n) / (Z_p + Z_p^{\text{out}})$, etc.). Same construction: central = upwind minus the impedance dissipation.
- Both exclude the fault face from the central set (DRDG3D node-based `ftype`: a face with 3 fault vertices is left upwind; SEAS `BuildCentralFluxFaceSet_` explicitly removes fault faces).

$\Rightarrow$ The central-flux **operator is faithful to Zhang/DRDG3D**. The instability is downstream of the kernel.

2.4 3. ROOT CAUSE – imaginary-axis stability of the time integrator $\times$ zero dissipation

Step 1 — central flux $\Rightarrow$ imaginary eigenvalues. On the central-flux corridor the semi-discrete DG operator loses its $+\frac{1}{2} |A_n| (Q^- - Q^+) \succeq 0$ dissipation and is energy-neutral (skew); the modes localized there have eigenvalues with $\approx$ zero real part, i.e. **purely imaginary $\lambda = i\omega$** . Upwind, by contrast, gives those modes a **negative** real part (the dissipation), placing them in the open left half-plane.

Step 2 — does the explicit one-step map grow an imaginary mode? That is decided solely by the integrator’s stability function $R(z)$ evaluated at $z = \lambda \Delta t = iy$:

- **ADER / Cauchy–Kovalevskaya of order O** (`wave_operator.hpp:417–449`: the predictor Taylor-expands $Q(t+\tau) = \sum_k \{k\} (\tau^k/k!) \cdot D(k)$ via the CK recursion $D(k) = L^k Q$): for the linear semi-discrete system its one-step amplification is the **truncated exponential** $R(z) = \sum_{k=0}^{\infty} z^k/k!$ — i.e. it is exactly the order-O Lax–Wendroff/Taylor scheme.
- **RK54** (DRDG3D, `mod_para.F90` RK54 default): 4th-order; $|R(iy)| \leq 1$ on a finite imaginary interval.

Evaluating $|R(iy)|$ on the imaginary axis:

integrator	O	$R(z)$	$ R(iy) \leq 1$ for	imaginary central-flux modes
ADER-O2 (SAFS)	2	$1 + z + z^2/2$	no $y > 0$	amplified at every $\Delta t \rightarrow$ blow-up

integrator	O	$R(z)$	$\ R(iy)\ \leq 1$ for	imaginary central-flux modes
ADER-O3 (TPV104)	3	$+z^3/6$	$0 < y \leq \sqrt{3} \approx 1.73$	$y > \sqrt{3}$ grow $\rightarrow$ localized spurious slip
ADER-O4	4	$+z^4/24$	$0 < y \leq 2.83$	stable iff CFL keeps $\ \lambda\ \Delta t < 2.83$
RK54 (DRDG3D)	4	5-stage LSRK	$0 < y \leq 2.8-3.4$	stable at Zhang's CFL 0.3

where $\|R(iy)\|^2 = 1 + y^4/4$ for O2 (exceeds 1 for **every** $y > 0$) and $1 - y^4/12 + y^6/36$ for O3 (exceeds 1 only for $y > \sqrt{3}$); RK54's imaginary-axis limit is at least the classic RK4 value $2\sqrt{2} \approx 2.83$.

This explains every observation:

- *Why SEAS blows up but DRDG3D does not, with the identical central kernel:* ADER-O2's $\|R(iy)\| > 1$ for all y ; RK54's ≤ 1 . The discriminant is the integrator, not the flux.
- *Why “smaller Δt still blew up — not a CFL trip” (§1, debug §1.2):* for $0 \leq 2$, $\|R(iy)\| > 1$ for **every** $y > 0$, so **no Δt exists** that stabilizes a purely-imaginary mode. The code's per-mode CFL knob (None 1.0 / Adjacent 0.9 / AllContinuous 0.4, `wave_operator.inl:5658–5663`, flagged as an “interim placeholder”) is the **wrong lever** at $0 \leq 2$ — tightening Δt only shrinks y , and $\|R(iy)\| > 1$ for all y .
- *Why TPV104 (O3) showed slow, point-source spurious slip while SAFS (O2) showed catastrophic global runaway:* O3 is stable for $y \leq \sqrt{3}$ — only the highest-frequency, near-Nyquist modes ($\|\lambda\| \Delta t > \sqrt{3}$) grow $\rightarrow$ localized hot spots; O2 amplifies **all** imaginary modes $\rightarrow$ global blow-up. The empirical severity ordering **O2** $\gg$ **O3** is exactly what the stability functions predict.

2.5 4. This corrects Candidate 1's framing

Candidate 1 (below) attributed the instability to the *cubic-in-time* ADER-O3 reconstruction and proposed re-running at **ADER-O2 as the “validated” fallback**. **That is backwards**. The mechanism is the truncated-exponential stability function on the imaginary axis, which is **monotonically worse as the order drops**: O2 is *unconditionally* amplifying, O3 only conditionally ($y \leq \sqrt{3}$), O4 better. Lowering the ADER order makes the central-flux instability **worse, not better** — and the SAFS O2+adjacent runaway is the empirical refutation of “O2 is the safe combination.”

2.6 5. Re-adjudication of the five 2026-04-28 candidates

#	candidate	2026-05-28 verdict
1	ADER-O3 $\times$ central	PROMOTED & RE-FRAMED → ROOT CAUSE. The discriminant is <i>imaginary-axis amplification of the order-O CK/Taylor stability function, worst at low order</i> (O2 unconditional). Not specific to the cubic term.
2	element- vs node-based face set ($\sim 2\times$ larger)	CONTRIBUTING, secondary. More central faces $\Rightarrow$ more imaginary-mode seeds and a wider non-dissipative corridor (241 089 faces) $\Rightarrow$ faster growth. But node-based + ADER-O2 would still blow up. Not decisive.
3	boundary sponge / PML	NOT the cause. DRDG3D's published-stable TPV5 runs <code>use_damp=0</code> ; SAFS blew up independent of reflections. Amplifier at most.
4	conservative ($\rho v, \epsilon$) vs non-conservative (σ, v) form	NOT the cause. Equivalent for the linear system; central kernels verified identical (§2).
5	friction-coupling cadence	NOT the cause. Fault face is upwind in both; the mechanism lives on the <i>fault-adjacent</i> bulk faces.

2.7 6. Remedies, re-ranked by the root cause

1. **mixed_flux = none (pure upwind)** — restores $+\frac{1}{2}|A_n|(Q^- - Q^+) \Rightarrow$ eigenvalues move into the open left half-plane $\Rightarrow$ ADER-O2 is stable there. Zero cost, already the SAFS default, and matches SeisSol (upwind everywhere). **Decisive fix for SAFS.**
2. If mixed flux is wanted for a $p \geq 2$ benefit: **(a)** match DRDG3D — advance the central-flux runs with an RK whose stability region contains the imaginary axis (RK4/RK54) at Zhang's CFL ≈ 0.3 ; or **(b)** raise `ader_order` ≥ 4 **and** tighten CFL so $\max|\lambda|\Delta t < 2.83$, **plus** a top-mode exponential filter to kill the residual $y > 2.83$ Nyquist modes. ADER-O3 alone is insufficient (the Nyquist mode exceeds $\sqrt{3}$).
3. **Restore a sliver of dissipation** on the central faces: α -blend $F = \alpha \cdot F_{\text{central}} + (1-\alpha) \cdot F_{\text{upwind}}$ with $\alpha \leq 1$ (or the §“Recommended next steps” filter). Any $\alpha < 1$ gives the modes a negative real part $\Rightarrow$ even ADER-O2 is stable. Cheapest way to keep most of the central-flux dispersion benefit while curing the imaginary growth.

4. **Match DRDG3D’s node-based face set** — shrinks the corridor but does **not** cure the mechanism; pair with 2 or 3, never alone.

2.8 7. Bottom line of the comparison

SEAS-MFEM is **faithful to Zhang/DRDG3D in the flux kernel and in keeping the fault face upwind**, but **unfaithful in the two robustness-determining choices**: (i) the time integrator (ADER-O2 vs RK54) — *decisive* — and (ii) the adjacent-face set (element- vs node-based) — *secondary*. DRDG3D’s central flux is stable because RK54’s stability region contains the imaginary axis; SEAS-MFEM’s algebraically-identical central flux blows up because ADER-O2’s does not. For SAFS at $p=1$ — where the central-flux fault-parallel dispersion benefit is negligible — the correct configuration is `mixed_flux = none`.

The original 2026-04-28 hypothesis-era analysis follows unchanged, as the supporting trail.

2.9 Architecture

Aspect	DRDG3D	SEAS-MFEM (current)
Conservative variables	($\rho v_x, \rho v_y, \rho v_z, \epsilon_{xx}, \epsilon_{yy}, \epsilon_{zz}, \epsilon_{yz}, \epsilon_{xz}, \epsilon_{xy}$) (9-vector, momentum + strain)	($\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}, v_x, v_y, v_z$) (9-vector, stress + velocity)
Form	Conservative $\partial_t Q + \nabla \cdot F(Q) = 0$	Non-conservative $\partial_t Q + A_x \partial_x Q + \dots = 0$ (P-system)
Polynomial order	Compile-time constant <code>pOrder</code> (default 2; <code>mod_para.F90</code> line 23-24)	Runtime <code>--order</code>
Time integrator	RK22 / RK33 / RK54 low-storage Williamson (<code>mod_para.F90</code> line 66, RK54 default; per RK substep one snapshot RHS evaluation)	ADER one-shot or substep (cubic-in-time Cauchy–Kowalewski predictor at order = <code>--ader-order</code>)
Mixed-flux selector	<code>flux_method 1 = AllContinuous, 2 = Adjacent</code> (default 2; <code>mod_read.F90</code> line 105)	<code>MixedFluxMode::None / Adjacent / AllContinuous</code> (default None)
Adjacent face set	Node-based : a face is “adjacent” if 1 or 2 of its 3 vertices lie on the fault (<code>matlab/set_fluxtype_from_faultnodes.m</code>)	Element-based : a face is “adjacent” if at least one of the two cells touching it is a fault element (<code>BuildCentralFluxFaceSet_wave_operator.inl</code>)

Aspect	DRDG3D	SEAS-MFEM (current)
Fault face flux	Always upwind / Godunov with α -coefficients, no central-flux dispatch on the fault face itself	Same — fault face always Godunov
Boundary stabilisation	Optional Gaussian sponge layer at domain edges, applied per-RK-substep (mod_damp.F90; use_damp flag, default 0)	PML (pml_layer.cpp) — different mechanism, also opt-in
Modal filter / limiter	None	None

2.10 Detailed walkthrough – DRDG3D’s central-flux dispatch

Entry point: subroutine get_flux(mesh, u, ie, qi, fluxs) in mod_wave.F90:347.

Per-face dispatch logic (mod_wave.F90:587–618)

```

if (flux_method == 2 .and.      &
    mesh%bctype(is, ie) == BC_IN .and. &
    mesh%fluxtype(is, ie) == 1) then
    fstar(1) = dTx
    fstar(2) = dTy
    fstar(3) = dTz
    fstar(4) = n_n(1) * dVx
    fstar(5) = n_n(2) * dVy
    fstar(6) = n_n(3) * dVz
    fstar(7) = n_n(3) * dVy + n_n(2) * dVz
    fstar(8) = n_n(3) * dVx + n_n(1) * dVz
    fstar(9) = n_n(2) * dVx + n_n(1) * dVy
    fstar = 0.5 * fstar
    goto 100
end if

```

Where $dTx = T(u_R) \cdot n_x - T(u_L) \cdot n_x$ and $dVx = v(u_R) \cdot \hat{x} - v(u_L) \cdot \hat{x}$, both built via extract_traction_velocity(mod_eqns.F90:151) with the neighbour’s material parameters when bimaterial (rho_out, cp_out, cs_out). n_n is the face unit normal in global coordinates; goto 100 skips the upwind α -coefficient block at lines 675-688.

Math: this is the lift form of the standard central flux for a conservative-form linear hyperbolic system,

$$F^* - F_L = \frac{1}{2} \cdot (F(u_R) - F(u_L)) \rightarrow \frac{1}{2} \cdot A_n \cdot (Q_R - Q_L)$$

decomposed by row: - rows 1-3 (momentum equations): $\frac{1}{2} \cdot (T_R - T_L)$ (jump of normal traction). - rows 4-9 (strain equations): $\frac{1}{2} \cdot \text{sym}(n \otimes (v_R - v_L))$ (symmetric outer product of normal $\times$ velocity jump). For row 4 only the diagonal $n_x \cdot (v_R \cdot \hat{x} - v_L \cdot \hat{x})$ survives; rows 7-9 carry the full symmetric cross-terms.

Sign convention: f_{star} is used in `mod_wave.F90:300+` as $\text{flux} - f_{\text{star}}$ (lift correction added to the L-side cell after partial integration). DRDG3D returns $F^* - F_L$, not F^* . Equivalent at infinite precision; only the convention differs from SEAS-MFEM.

Adjacent-face set construction (`set_fluxtype_from_faultnodes.m`)

```
node_flag = zeros(Nnode,1);
node_flag(fnodes) = -1;           % fault vertices
fn = node_flag(elem(FTov',:));    % per-face per-vertex flag (3 × 4 × Nelem)
ftype = sum(fn, 1);               % per-face sum over its 3 vertices
% ftype ∈ {0, -1, -2, -3} for {0,1,2,3} fault vertices on the face

ftype1(ftype == -1) = 1;          % 1 fault vertex → central flux
ftype1(ftype == -2) = 1;          % 2 fault vertices → central flux
% ftype == 0 (no fault vertex) → upwind (default 0)
% ftype == -3 (face IS the fault face) → upwind (the fault dispatch handles it)
```

So DRDG3D’s “adjacent” set covers faces that **touch the fault at one or two vertices** but are not the fault face themselves. This is a RING of sibling faces around each fault face: each fault face sits in a prism of $2 \times (3 + 1) = 8$ adjacent faces (3 sibling faces on each tet, plus the fault face itself which is upwind).

SEAS-MFEM equivalent (`wave_operator.inl::BuildCentralFluxFaceSet_`)

SEAS-MFEM picks the face set differently:

```
if (mixed_flux_mode_ == MixedFluxMode::Adjacent) {
    for each interior face f {
        if (Elem1(f) is a fault-touching element ||
            Elem2(f) is a fault-touching element) {
            central_flux_face_set_.insert(f);
        }
    }
}
```

i.e. **element-based**: any face on the boundary of a fault-element gets central. This is a STRICTLY LARGER set than DRDG3D’s node-based selection — every face DRDG3D includes is also in SEAS-MFEM’s set, plus extras on the far side of fault elements (the inner three faces of each fault tet that share at most 1 vertex with the fault face).

Numerical flux: `GodunovFlux::Central` (SEAS-MFEM, `godunov_flux.cpp:382–423`)

```
// F_rot = 0.5 · (Ax_plus_ + Ax_minus_) · (Q_self_rot + Q_nbr_rot)
real_t Q_sum[NUM_STATE];
for (int i = 0; i < NUM_STATE; i++) Q_sum[i] = Q_self_rot[i] + Q_nbr_rot[i];
for (int i = 0; i < NUM_STATE; i++) {
    real_t s = 0.0;
    for (int j = 0; j < NUM_STATE; j++)
```



```

        s += (Ax_plus_(i, j) + Ax_minus_(i, j)) * Q_sum[j];
        F_rot[i] = 0.5 * s;
    }
    T.Mult(F_rot, F_h); // rotate back to global

```

This is $F^* = \frac{1}{2} A_n (Q_L + Q_R)$ — equivalent to the central flux for a non-conservative linear hyperbolic system. The L+R sum is correct, the eigenvalue split $Ax_{plus} + Ax_{minus} = A_x$ is only correct if all non-zero eigenvalues' rank-1 outer products are summed. Linear elasticity has 9 characteristics: 3 with eigenvalue $\pm c_p$, 6 with $\pm c_s$ (triple multiplicity each), and three zero eigenvalues (the static modes orthogonal to the face). The standard implementation puts the zero-eigenvalue projector with weight 1 in either A_{plus} or A_{minus} , or splits it. Whether SEAS-MFEM and DRDG3D handle these zero modes identically would require checking SEAS-MFEM's `BuildSplitMatrices_`; they should agree because in central-flux mode $A_{plus} + A_{minus} = A$ regardless of the convention used to split the kernel.

2.11 Call graph (DRDG3D mixed-flux path, ADER-equivalent)

```

seis3d.F90::main_loop
  L→ do irk = 1, nrk                                     ! RK54 substep
    |→ rhs(mesh, u, qi, ru)                             ! mod_wave.F90:73
    |    L→ get_flux(mesh, u, ie, qi, fluxs) ! mod_wave.F90:347
    |        |→ extract_traction_velocity(uL, ...)
    |        |→ extract_traction_velocity(uR, ...)
    |        |→ if (flux_method==2 .and. fluxtype==1) → CENTRAL (line 587)
    |        |→ if (flux_method==1 .and. BC_IN)        → CENTRAL (line 604)
    |        L→ else                                    → UPWIND/Godunov (line 675)
    |            L→ if (bctype >= BC_FAULT)              → friction Riemann (line 697)
    |→ tu = rk4a(irk)*tu + dt*hu                        ! Williamson update
    |→ u = u + rk4b(irk)*tu
    |→ if (use_damp == 1)                                ! mod_damp.F90 sponge zone
        do i = 1, Nvar; u(:,i) = u(:,i) * mesh%damp; end do

```

vs SEAS-MFEM's ADER substep dispatch:

```

WaveOperator::AdvanceADER (or AdvanceADERWithSubStep for substep iterator)
  |→ ComputeADERSubStepStates(Q, dt, ader_order, tau_nodes, Q_per_node)
  |    L→ Cauchy-Kowalewski cubic predictor in time (degree = ader_order)
  |→ Per macro step (or per substep):
  |    |→ ComputeADERFaceFluxRHS(...)                    ! wave_operator.inl
  |        |→ For each interior face f:
  |            |    if (mf_on && central_flux_face_set_.count(f))
  |                |    flux_.Central(nor, Q_self, Q_nbr, F_h) ! godunov_flux.cpp:382
  |                |    else
  |                    |    flux_.Interior(nor, Q_self, Q_nbr, F_h) ! upwind / Godunov
  |            L→ Friction face dispatch via FaultFaceFlux::EvaluateADER / EvaluateADER_LSW
  L→ No sponge / damp post-step (PML is the equivalent, also opt-in)

```

2.12 Conventions

- DRDG3D loop variable layout: `mesh%vmapM(i, is, ie)` = local-vertex / face-quadrature index for "minus" side, `mesh%vmapP(i, is, ie)` = ditto for "plus" side. Indexing $N_{fp} = (Order+1)(Order+2)/2$ per face.
- DRDG3D's `direction(is, ie)` encodes the orientation rotation between L and R sides on a non-conforming face; uses `flipped_index` lookup table.
- Material parameters per element: `mesh%rho(ie)`, `mesh%vp(ie)`, `mesh%vs(ie)`.
- Fault parameters per face-quadrature point: `mesh%Tau0n(i, is, ief)`, `mesh%mu_s(i, is, ief)`, `mesh%Dc(i, is, ief)`, ...
- Friction-law selector is integer `friction_law` in `[0..4]`: 0=LSW (default), 1=ageing-law RS, 2=slip-law RS, 3=slip+flash heating, 4=time weakening.
- Sign convention on fault: $Tau_n = fL(1) + fstar(1) + Tau0n$ — TOTAL normal traction is "fluctuation trial + corrected jump + background pre-stress" (same convention SEAS-MFEM uses with the v9.4.0 channel split).

2.13 Gotchas

1. **Polynomial order is COMPILE TIME** (`#define pOrder 2` default in `mod_para.F90` line 23). Changing order → recompile. SEAS-MFEM allows runtime `--order`.
2. **Default `flux_method = 2`** — DRDG3D is published with mixed-flux ADJACENT ON by default. The TPV5 example's `parameters.yaml` doesn't override it; published TPV5 results use mixed-flux out of the box.
3. **Default `use_damp = 0`** — the Gaussian sponge is OFF by default. Published TPV5 results don't use it either. So DRDG3D's stable mixed-flux comes WITHOUT additional boundary damping.
4. **Time integrator default is RK54** (`timeIntegrationMethod = 1`, Kennedy-Carpenter low-storage 5-stage 4th-order). Per RK substep, a SINGLE snapshot RHS evaluation. No predictor that reconstructs Q polynomial in time.
5. **Adjacent-face set is node-based, not element-based** — STRICTLY SMALLER than SEAS-MFEM's element-based set by a factor of $\sim 2\times$ (excludes inner-tet faces that share fewer than 2 vertices with a fault face).
6. **Both fault-face flux paths in DRDG3D have the central-flux block COMMENTED OUT** (`mod_wave.F90:663–688` in `get_flux`, lines 1362–1387 in `get_flux_dd`). Mixed-flux's central-flux dispatch never applies to the FAULT FACE — only to fault-adjacent faces. This matches SEAS-MFEM's behaviour but is worth confirming.

2.14 Differential analysis – what is SEAS-MFEM doing differently?

Five candidates, in decreasing likelihood of explaining the late-time spurious-rupture symptom:

Candidate 1 (most likely): ADER-O3 cubic-in-time predictor $\times$ central-flux interaction

DRDG3D uses RK54 — per substep, the RHS is evaluated on a SNAPSHOT state $u(t_{\text{substep}})$. Time accuracy is achieved through 5-stage Runge-Kutta mixing, NOT through a polynomial-in-time reconstruction.

SEAS-MFEM uses ADER's Cauchy-Kowalewski recursion to BUILD a polynomial $Q(\tau)$ of degree `ader_order` IN TIME on each macro step, then averages or evaluates that polynomial at substep nodes for the corrector. With `ader_order = 3`, the predictor reconstructs cubic-in-time content per

macro step. That cubic mode is fed into the RHS, which on a central- flux face is a NON-DISSIPATIVE flux: the high-frequency content has nowhere to go.

RK54 has no analogous high-mode reconstruction — each substep’s RHS sees only the current snapshot, with whatever spatial dissipation the flux provides. Net dissipation per macro step in RK54 is the Williamson scheme’s stability-weighted average; in ADER-O3 it’s the zeroth-degree term of the time polynomial only. The HIGHER-ORDER modes are exactly what central flux fails to dampen.

Test: re-run TPV104 P=2 + `--mixed-flux adjacent + --ader-order 2` on the production mesh + 500 ranks. R-1204 already labels this as the “validated” combination. If it stays clean past $t = 6.5$ s, ADER-O3 $\times$ central-flux is the root cause.

Candidate 2: Element-based vs node-based face set

SEAS-MFEM’s “Adjacent” set is strictly larger than DRDG3D’s. The extra faces are the inner three faces of each fault tet that share fewer than 2 vertices with the fault face — i.e., faces deep in the tet’s volume. Central flux on those inner faces cuts a wider swath of non-dissipative DG flux around each fault element. More central-flux faces $\rightarrow$ more chances for under-damped modes to propagate.

Test: compare central-flux face counts at the same mesh: DRDG3D’s set should be $\sim 1/2$ of SEAS-MFEM’s. Tighten SEAS-MFEM’s `BuildCentralFluxFaceSet_` to match DRDG3D’s node-based criterion and re-run.

Candidate 3: Boundary sponge

DRDG3D has the option of a Gaussian sponge layer (`use_damp = 1`). If the published TPV5 stability relies on it (need to confirm), our PML is supposed to be the equivalent but interacts with mixed-flux ADJACENT differently. DRDG3D’s sponge multiplies $u(\cdot)$ by a per-DOF factor every RK substep — a robust low-pass on the bulk field at the domain edge that prevents spurious reflections.

Test: compare published DRDG3D TPV5 results with `use_damp = 0` vs `use_damp = 1`; if the symptom only appears with `use_damp = 0` plus mixed-flux + RK54, the boundary stability is necessary. (However, `parameters.yaml` defaults `use_damp = 0`, suggesting the published results don’t need it.)

Candidate 4: Conservative (p, ϵ) form vs non-conservative (v, σ) form

Mathematically equivalent at infinite precision. In FINITE-precision DG, the non-conservative form requires careful eigendecomposition for the zero-eigenvalue subspace of A_n ; the conservative form is self-consistent because the flux IS the physical traction & strain.

This is unlikely to be the root cause because both forms are correct at the level of long-time stability for a LINEAR hyperbolic system. But the floating-point error budget differs.

Candidate 5: Friction-coupling timing

DRDG3D’s fault-face flux is the SAME upwind/Godunov regardless of the mixed-flux setting — the central-flux dispatch never applies to the fault face, only to its neighbours. SEAS-MFEM’s substep iterator similarly keeps the fault face on the friction Riemann path. So both codes preserve the friction face flux unchanged when mixed-flux toggles. Differential: SEAS-MFEM’s iterator runs LSW or RS friction PER SUB-STEP inside the macro step, with the central flux feeding the bulk Q predictor between substeps. DRDG3D updates the fault state PER RK SUBSTEP from a snapshot Q. Different coupling cadence; whether it matters depends on whether the spurious modes propagate

fast enough to reach the fault between substeps.

2.15 Open questions

1. **What is $A_{x_plus_} + A_{x_minus_}$ in SEAS-MFEM's eigenvalue split when integrated against a P-system zero mode?** Need to read `BuildSplitMatrices_` to confirm $(A_{plus} + A_{minus})$ actually equals A_x including the kernel projection.
2. **Does DRDG3D's RK54 + mixed-flux survive $p \geq 3$?** `pOrder` is compile-time; would need to recompile DRDG3D with `pOrder = 3` and run the same `TPV5 + flux_method = 2` to settle whether the $ADJACENT \times p \geq 3$ combination is unstable in DRDG3D too. If DRDG3D blows up at $p=3$ + mixed-flux + RK4, the issue is intrinsic to mixed-flux at high order — not specific to ADER.
3. **What's the spectral content of the central flux's spurious mode?** A Fourier analysis on a 1-D linear-elasticity-system test would show the dispersion-dissipation characteristic of $\frac{1}{2} A_n (Q_L + Q_R)$ vs upwind. Both ADER-O3 and central flux are well-studied individually; the COMBINATION is less so.
4. **Does our element-based "Adjacent" set include faces where DRDG3D's node-based set excludes them, AND is one of those extra faces where the spurious hot spot first appears?** Worth a static check on a small mesh: print which faces are "extra" in SEAS-MFEM's set, then visualize.

2.16 Recommended next steps for the SEAS-MFEM TPV104 instability

In order of cost vs information yield:

1. **Re-run TPV104 $P=2$ + ADER-O2 + `--mixed-flux adjacent` on the production mesh.** This is the R-1204 "validated" combination and isolates Candidate 1 ($ADER-O3 \times ADJACENT$). One sbatch run.
2. **Tighten `BuildCentralFluxFaceSet_` to match DRDG3D's node-based criterion** (only include a face if $1 \leq \text{shared-vertices-with-a-fault-face} \leq 2$; exclude inner-tet faces that share 0 vertices with any fault face). Test if the spurious hot spot disappears on the same O3 + adjacent configuration. This isolates Candidate 2.
3. **Add a modal/exponential filter on bulk Q** as a safety net — a 4th-order exponential filter applied after each ADER step, only on faces in `central_flux_face_set_`'s neighbourhood, with cutoff at the top 10% of the spectrum. This is the standard fix for under-damped DG schemes (Hesthaven & Warburton Ch. 5). Cost: one filter assembly + one matrix-vector per step. Reversible (sets filter coefficient to 0).
4. **Add a one-sided alpha-blend** between Central and Interior for the dispatch on the central-flux face set: $F^* = \alpha \cdot \text{Central} + (1 - \alpha) \cdot \text{Interior}$, with α tunable in $[0, 1]$. $\alpha = 1$ is current behaviour; $\alpha = 0.95$ retains 95% of the central-flux benefit while restoring 5% upwind dissipation. This is a commonly-used regulariser in the LDG / mixed-flux literature.

The cheapest discriminator is (1). Suggest doing that first.